

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 11 of 22 — Core Analysis: H1 Test & Border Distance
analyze_results.py, compute_border_distance.py — Poori Line
By Line

Sakshi ke liye, Sakshi ki apni research par. PDF 6 se 10 tak humne dekha ki data kaise khinchte hain — satellite se, geocoding se, cloud-mask se. Ab, is PDF se, 'Core Analysis' phase shuru hoti hai: wahi khinchi hui data lekar, poochna 'kya isme ek asli, statistically-meaningful signal hai?' Ye do files is poore project ke sabse zyada baar-jaanchi-gayi, sabse zyada history rakhne wali files hain — inhi se is study ka sabse famous reversal (154 se 251 villages) aaya.

analyze_results.py is study ka core hypothesis-test chalata hai — kya VVP-I ke baad, satellite se dikhne wala built-up-area ya raat ki roshni significantly badhi? compute_border_distance.py har gaon ki international-border se doori nikaalta hai — jo aage H3 (border-proximity hypothesis) ke liye zaroori hai. Dono files chhoti hain, lekin dono ke peeche genuinely dramatic history hai.

Shuru Karte Hain — Core Analysis Ka Matlab

Ab tak, PDF 6 se 10 tak, humne sirf *data ikattha* kiya — coordinates, NDBI values, lights values, control-group, SAR, Dynamic World, buildings, sab kuch. Lekin raw numbers apne aap mein kuch nahi bolte. Agar ek gaon ka NDBI 2021 mein -0.25 tha aur 2025 mein -0.24, kya ye 'badlaav' hai, ya sirf measurement-noise? Isi sawaal ka jawaab dene ke liye statistics chahiye — wahi Wilcoxon test, p-value, Spearman correlation jo PDF 3 mein foundational level par sikhaye gaye the. Ye PDF unhi concepts ko, is study ke apne, asli data par, line-by-line apply karke dikhata hai.

EXAMPLE — Poore Class Ka Result Announce Karne Se Pehle

Socho tu ek teacher hai, aur tune apni poori class ka before-after test liya hai — kisi naye teaching-method ke baad. Har student ka score thoda-bahut badla hoga, kuch upar kuch neeche, sirf random variation se. Tu seedha ye nahi keh sakti 'method kaam kar gaya' sirf isliye ki average score thoda upar gaya — tujhe ek statistical test chahiye jo bataye ki ye badlaav genuinely meaningful hai ya sirf chance se ho sakta tha. `analyze_results.py` exactly yahi karta hai, 251 'students' (villages) ke liye, ek 'before' score (2021 NDBI) aur ek 'after' score (2025 NDBI) ke saath.

History Ek Nazar Mein — Ek Number Jo Teen Baar Palta

Is PDF ki sabse badi kahani ye hai ki summer-window ka H1 result, is poore project mein, teen baar reverse hua — pehle full-year null se summer significant (Entry 5, sirf 154 villages ke saath, kyunki Sikkim summer mein poori tarah drop ho gaya tha), fir wahi significant result Entries 15-20 tak 'sabse strong surviving signal' maana gaya, fir Entry 21 mein shak hua ki ye 154-village sample khud ek stale-archive artifact ho sakta hai, aur aakhir mein Entry 22 mein — jab Sikkim wapas aaya aur poori 251-village sample fresh, same-day extract hui — result wapas null ho gaya. Hum ye poori kahani, code ke exact us hisse mein dekhenge jahan sample-size decide hoti hai.

Part A — analyze_results.py: Is Study Ka Core Hypothesis Test

Docstring — Ek Consolidation Ki Kahani

```
"""
```

```
Core stats for RQ1 (built-up change) and RQ2 (state budget correlation),  
run against whichever compositing window is selected.
```

```
Consolidated on 2026-09-17 from two near-duplicate scripts (analyze_results.py  
and analyze_results_fullyear.py) that differed only in which window's paths  
they read/wrote and their print labels -- the same class of window-consistency  
bug this project's Development Log has caught before, now handled the same  
way did_model.py already handles it: one script, a --window flag, and a path  
dictionary per window instead of copy-pasted logic. See archive/README.md.
```

```
"""
```

Purpose: Ye docstring pehle line mein hi is file ka poora kaam bata deta hai — RQ1 (kya built-up-area badla) aur RQ2 (state-budget se correlation), jo bhi compositing-window chuni jaaye uske liye. Doosra paragraph ek zaroori, genuinely interesting history batata hai — ye ek engineering-hygiene kahani hai, koi statistical bug nahi. Pehle do alag files thi: analyze_results.py sirf summer window ke liye, analyze_results_fullyear.py full-year ke liye — dono mein *almost* identical code, sirf file-paths aur print-labels alag. Ye copy-paste pattern khud khatarnaak hai — agar kal kisi ek file mein bug-fix ho, doosri mein bhoolna asaan hota hai (bilkul wahi 'window-consistency' risk jo is poore project mein pehle bhi dikh chuka hai). 2026-09-17 ko, dono files ko ek script mein consolidate kiya gaya — ek --window flag, aur ek path-dictionary per window — bilkul wahi pattern jo did_model.py (PDF 12 mein aayega) pehle se hi use kar raha tha. Consolidate karne se pehle, purani file ka output byte-for-byte verify kiya gaya (md5sum se) naye script ke output ke against, taaki consolidation khud koi naya bug na le aaye — purani file archive/superseded_scripts/ mein rakhi gayi, delete nahi ki gayi. Syntax: Multi-paragraph triple-quoted docstring, jaisa poore project mein consistent.

EXAMPLE — Consolidation Kyun Ek 'Bug Fix' Jaisi Hi Zaroori Hai, Bina Kisi Bug Ke

Ye is project ka ek genuinely alag tarah ka 'history' hai — abhi tak humne jitni kahaniyaan dekhi hain, unme ek galti thi jo baad mein pakdi gayi. Yahan koi galti nahi hui thi — dono files sahi kaam kar rahi thi. Lekin do near-identical files rakhna, apne aap mein, ek *future* bug ka risk hai — jaisa is poore Development Log mein kai jagah dikha hai (jaise PDF 8/9 mein dekha, jab ek script fix hoti hai aur uski 'twin' file bhoolke fix nahi hoti). Isliye is project ka apna, established practice hai: jaise hi ek aisa pattern dikhe, use consolidate kar do, is se pehle ki wo genuinely diverge ho jaaye aur ek asli bug ban jaaye.

Imports Aur Path Dictionaries

```
import argparse

import pandas as pd
from scipy import stats

INPUT_PATHS = {
    "summer": "data/processed/border_optics_village_results_summer.csv",
    "full_year": "data/processed/border_optics_village_results.csv",
}
OUTPUT_PATHS = {
    "summer": "data/processed/border_optics_village_results_summer_analyzed.csv",
    "full_year": "data/processed/border_optics_village_results_analyzed.csv",
}
WINDOW_LABELS = {
    "summer": "SUMMER-MATCHED COMPOSITE WINDOW",
    "full_year": "FULL-YEAR COMPOSITE WINDOW",
}
```

Purpose: argparse command-line se --window flag padhne ke liye. pandas table-operations. from scipy import stats — PDF 9 mein dekha gaya, statistical tests ke liye. Teen dictionaries, ek-ek key per window ("summer", "full_year"): INPUT_PATHS batata hai kaunsi raw-extraction file padhni hai, OUTPUT_PATHS batata hai processed result kahan save karna hai, WINDOW_LABELS sirf console-print ke liye ek human-readable naam. Yahi teen dictionaries hain jo docstring ke 'path dictionary per window' wale vaade ko poora karti hain. Syntax: Do imports, teen dictionary literals — har ek do keys (window-naam se), values strings hain.

```
BUDGET_DATA = {
    "Arunachal Pradesh": {"projects": 2082, "budget_cr": 2749.74},
    "Uttarakhand": {"projects": 200, "budget_cr": 270.58},
    "Sikkim": {"projects": 63, "budget_cr": 188.90},
}
```

Purpose: Ye ek naya, nested dictionary hai — RQ2 (budget-vs-outcome correlation) ke liye zaroori, official data. Har state ke against, uska VVP-I project-count aur sanctioned-budget (crore rupees mein). Dhyaan do — sirf teen states hain yahan, Himachal Pradesh nahi — kyunki core sample (jo aage is_core_sample filter se define hoga) sirf inhi teen states ki villages rakhta hai; Himachal ki 7 villages 'illustrative' hain, core statistical tests mein shaamil nahi. Syntax: Nested dictionary — bahar state-naam keys, andar do keys ("projects", "budget_cr") har state ke liye. Alignment-spaces (extra spaces state-naam ke baad) sirf readability ke liye, Python inhe ignore karta hai.

main() Part 1 — Data Load Karna Aur Change-Metrics Banana

```
def main(window):
    input_path = INPUT_PATHS[window]
    output_path = OUTPUT_PATHS[window]

    df = pd.read_csv(input_path)

    # --- Derived change metrics ---
    df["ndbi_change"] = df["ndbi_after"] - df["ndbi_before"]
    df["lights_change"] = df["lights_after"] - df["lights_before"]

    core = df[df["is_core_sample"] == True].copy()
```

Purpose: Dictionary se sahi input/output path nikaalna, window ke hisaab se. CSV padhna. Fir do naye columns banaye jaate hain — `ndbi_change` aur `lights_change` — seedha subtraction se, poore dataframe par ek saath. Ye is study ka sabse fundamental derived-quantity hai: 'after minus before,' jo H1 ka poora aadhaar hai. Aakhir mein, core banaya jaata hai — sirf wo rows jinka `is_core_sample` True hai (matlab, Arunachal/Sikkim/Uttarakhand ki 251 villages, Himachal ki 7 illustrative villages nahi). Syntax: Function-definition, ek parameter (`window`). Dictionary-lookup se paths nikaalna. Vectorized subtraction — `df["col"] = df["a"] - df["b"]`, poore column par ek saath, row-by-row loop ki zaroorat nahi. Boolean-mask filtering + `.copy()` — `.copy()` zaroori hai taaki core apna khud ka, independent dataframe ho, `df` ka sirf ek 'view' na ho (warna aage core mein kiya gaya koi bhi badlaav, ek confusing warning de sakta tha).

Descriptive Stats — Pehle Dekho, Fir Test Karo

```
print("NDBI change (built-up index, after - before):")
print(core["ndbi_change"].describe())
print()
print("VIIRS night-light change (radiance, after - before):")
print(core["lights_change"].describe())
print()

print("By state:")
print(core.groupby("state")[["ndbi_change", "lights_change"]].agg(["mean", "median", "std"]))
```

Purpose: Kisi bhi formal test se pehle, ye block sirf data ko 'dekhta' hai — `.describe()` ek column ka count, mean, std, min, quartiles, max ek saath deta hai, ek quick sanity-check ke liye. Fir `groupby("state")` se, har state ke liye alag-alag mean/median/std dikhaya jaata hai — taaki koi bhi ek state, aage aane wale pooled-test ko akela drive na kar raha ho, ye pehle hi visible ho jaaye. Syntax: `.describe()` — pandas Series ka built-in summary-statistics method. `.groupby("state")[["col1, col2]].agg(["mean", "median", "std"])` — pehle state ke hisaab

se groups banao, fir do specific columns chuno, fir har group ke liye teen alag aggregation-functions ek saath chalao — result ek multi-level table hota hai.

RQ1/H1 — NDBI Ka Wilcoxon Test, Aur Yahin 154-Village Kahani Shuru Hoti Hai

```
valid_ndbi = core.dropna(subset=["ndbi_before", "ndbi_after"])
print(f"NDBI test run on {len(valid_ndbi)}/{len(core)} villages with valid data "
      f"(rest excluded -- no cloud-free imagery in the {window.replace('_', '-')} window)")
w_stat, w_p = stats.wilcoxon(valid_ndbi["ndbi_after"], valid_ndbi["ndbi_before"],
                             alternative="greater")
print(f"Wilcoxon statistic = {w_stat:.2f}, p = {w_p:.6f}")
```

Purpose: Yahi wo exact line hai — `core.dropna(subset=["ndbi_before", "ndbi_after"])` — jahan is study ka sabse famous number decide hota hai: kitni villages ke paas *valid* NDBI data hai. `len(valid_ndbi)` wahi number hai jo kabhi 154 tha, ab 251 hai. Fir `stats.wilcoxon(after, before, alternative="greater")` — PDF 9 mein dekha gaya wahi ek-sided signed-rank test, jo poochta hai 'kya after values, before se significantly zyada hain.' Syntax: `dropna(subset=[col1, col2])` — missing-value removal, dono columns check karke. F-string mein `len(valid_ndbi)` aur `len(core)` dono embed karna, ek 'X/Y' format mein. `window.replace('_', '-')` — string method, "full_year" ko "full-year" mein badalta hai, sirf display ke liye. `stats.wilcoxon(...)` tuple return karta hai — statistic aur p-value.

EXAMPLE — Entry 5 → Entries 15-20 → Entry 21 → Entry 22 — Ek Number Ki Poori Zindagi

Ye `len(valid_ndbi)` ka number, is poore project ke sabse dramatic arc ka nayak hai. Entry 4 mein, jab full-year composite use hua, saari 251 core-sample villages valid thi, aur result null tha ($p=1.000$). Entry 5 mein, jab snow-confound hatane ke liye summer-only (June-September) window try kiya gaya, ek naya problem mila: Sikkim ki saari 31 villages, us narrow window mein, zero cloud-free images de rahi thi (peak monsoon, Eastern Himalaya mein) — matlab, wo `dropna()` se hat gayi, aur `valid_ndbi` sirf 154 villages ki reh gayi. Isi 154-village sample par, result ekdum flip ho gaya: highly significant increase ($p<0.000001$). Entries 15 se 20 tak, ye 154-village summer result is study ka 'sabse strong surviving signal' maana gaya — leave-one-out, randomization-inference, log-transform, sab robustness-checks isi 154-village result par chale, aur sab pass hue. Lekin Entry 21 mein ek shak uthaya gaya: jab isi date-range ka ek doosra extraction (multi-year script, baad mein chalaya gaya) check kiya gaya, Sikkim ki wahi 31 villages, is baar *valid* scenes dikha rahi thi (24 aur 31 images) — matlab, 'Sikkim ka zero-images' hona, ek permanent geography-fact nahi, balki ek us-din-ka archive-snapshot artifact tha (Sentinel-2 ka archive, purane dates ke liye bhi, samay ke saath scenes backfill karta rehta hai). Entry 22 mein, treated aur control, dono ka ek fresh, same-day re-extraction hua — aur is baar saari 251 villages valid thi (Sikkim samet). Naya result: $p=0.435831$ — wapas null. Ye teen-stage reversal (null → significant-on-incomplete-sample → null-on-complete-sample) is poore project ka sabse zaroori single lesson hai: sample-size sirf ek number nahi hai, ye khud data-quality ka ek signal hai, aur usse hamesha poochna chahiye 'kyun itni villages missing hain.'

```

valid_lights = core.dropna(subset=["lights_before", "lights_after"])
print(f"Lights test run on {len(valid_lights)}/{len(core)} villages with valid data")
w_stat_l, w_p_l = stats.wilcoxon(valid_lights["lights_after"],
valid_lights["lights_before"], alternative="greater")
print(f"Same test on VIIRS night-lights: statistic = {w_stat_l:.2f}, p = {w_p_l:.6f}")

```

Purpose: Bilkul wahi test, ab VIIRS night-lights ke liye — apna alag dropna(), apna alag Wilcoxon call. Interesting baat: night-lights, poore is study mein, kabhi bhi Sikkim-ki-wajah-se-sample-chhota nahi hua — VIIRS monthly composite pehle se hi 'cloud-cleaned' hota hai (naam mein khud 'VCMSLCFG' — cloud-mask — hai, jaisa PDF 8 mein dekha), isliye lights hamesha saari 251 villages ke liye available raha, chahe NDBI ka sample size kuch bhi ho. Yahi wajah hai ki poore is project mein, night-lights ko 'zyada temporally-stable proxy' maana gaya — Entry 5 se hi. Syntax: Identical pattern jo NDBI ke liye upar dekha, sirf column-naam badal kar (lights_before/lights_after).

RQ2 — Budget Vs. NDBI Change, Ek Honest 'n=3' Caveat Ke Saath

```

state_summary = valid_ndbi.groupby("state")[["ndbi_change",
"lights_change"].mean().reset_index()
state_summary["projects"] = state_summary["state"].map(lambda s:
BUDGET_DATA[s]["projects"])
state_summary["budget_cr"] = state_summary["state"].map(lambda s:
BUDGET_DATA[s]["budget_cr"])
state_summary["n_villages_with_ndbi"] =
state_summary["state"].map(valid_ndbi["state"].value_counts())

```

Purpose: Har state ka mean NDBI/lights-change nikaalna (valid_ndbi se, matlab sirf jinke paas valid data hai unhi villages se), fir .reset_index() se groupby ka result ek normal dataframe mein wapas lana. Fir teen naye columns, har ek .map() se banaya jaata hai — state-naam ko BUDGET_DATA dictionary mein dekh kar projects/budget nikaalna, aur value_counts() se har state mein kitni valid-NDBI villages hain, ye count karna. Syntax: .groupby(...).mean().reset_index() — groupby ke baad index state-naam ban jaata hai; .reset_index() use wapas ek normal column bana deta hai. .map(lambda s: BUDGET_DATA[s]["projects"]) — ek chhota, inline anonymous function (lambda), jo har state-naam s ko uske budget-dictionary ki value se replace karta hai. .map(series.value_counts()) — ek doosre column ki value-counts (khud ek Series, state-naam se indexed) ko lookup-table ki tarah use karna.

```

if len(state_summary) >= 3:
    rho, p_val = stats.spearmanr(state_summary["budget_cr"],
state_summary["ndbi_change"])
    print(f"Spearman correlation (budget vs mean NDBI change): rho = {rho:.3f}, p =
{p_val:.3f}")
    print("NOTE: with only 3 states (or fewer, if data was missing for one), this p-value")
    print("is not meaningful on its own -- report the direction/pattern descriptively.")
else:

```

```
print(f"Only {len(state_summary)} state(s) have valid NDBI data -- correlation not computable.")
```

Purpose: Ye block khud is study ki intellectual-honesty ka ek achha example hai. Spearman correlation sirf tabhi compute hota hai jab kam-se-kam 3 states ke paas valid data ho (if `len(state_summary) >= 3`) — aur jab compute hota bhi hai, code khud, apne print-statement mein, ek loud warning deta hai: 'sirf 3 data-points ke saath, ye p-value meaningful nahi hai, sirf direction/pattern ko descriptively report karo.' Ye statistically sahi hai — ek Spearman test, sirf teen points par, formally power-nahi-rakhta, chahe rho kitna bhi bada dikhe. Agar 3 se kam states ke paas data ho (jaisa 154-village-era mein hota tha, jab Sikkim drop ho jaata tha), code seedha bata deta hai ki correlation compute hi nahi ho sakta, koi misleading number nahi deta.

Syntax: `if len(...) >= 3: ... else: ...` — conditional branch. `stats.spearmanr(x, y)` — do arrays ka rank-correlation, tuple return karta hai. F-string mein hardcoded warning-text, statistical honesty ke liye jaan-bujh kar likha gaya.

Enriched Dataset Save Karna — Ek Chhota, Practical CRLF Detail

```
df.to_csv(output_path, index=False, lineterminator="\r\n")
print(f"\nSaved enriched dataset to {output_path}")
```

Purpose: File save karna, jisme ab `ndbi_change`/`lights_change` columns bhi shaamil hain (poore df par, sirf core par nahi — matlab Himachal ki illustrative villages bhi save hoti hain, sirf unpe statistical test nahi chalta). `lineterminator="\r\n"` ek chhota, lekin practical detail hai jiska apna comment hai code mein: `data/processed/` ki har doosri CSV file, originally CRLF (Windows-style line-endings) ke saath save hui thi. Agar ye specify na kiya jaaye, pandas apne Linux-default (sirf LF) se save karega — jo file ke content mein koi asli badlaav nahi karta, lekin git diff mein poori file 'badli hui' dikhegi (har line ka ending alag), jo real changes ko dhoondhna mushkil bana deta.

Syntax: `.to_csv(path, index=False, lineterminator="\r\n")` — `index=False` row-numbers ko file mein na likhna, `lineterminator` explicitly line-ending-style set karna, os-default ke bharose reh jaane ki jagah.

__main__ Block

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Core RQ1/RQ2 stats for BORDER OPTICS, run against a single
        compositing window."
    )
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()
    main(args.window)
```

Purpose: Standard `--window` flag, jaisa poore acquisition-module mein dekha. Ye file sirf ek baar chalti hai per window, isliye ise dono windows ke liye alag-alag chalana padta hai (`--window summer` aur `--window full_year`, dono baar). **Syntax:** Familiar argparse pattern.

Final State — H1 Ka Current, Corrected Verdict

Ye hai is file ka current, 100%-corrected output — Entry 22 ke same-day, complete re-extraction ke baad, aur us se pehle ka koi bhi 154-village-era number ab historical record hai, current claim nahi.

Window	Metric	n (valid)	Wilcoxon p	Verdict
Full-year	NDBI change	251/251	1.000000	Null
Full-year	Lights change	251/251	0.050075	Borderline null
Summer	NDBI change	251/251	0.435831	Null (post-Entry-22)
Summer	Lights change	251/251	0.999994	Null

Chaaron combinations null (ya borderline-null) hain — koi bhi window, koi bhi proxy, ek significant 'built-up-area badha' ya 'lights badhi' result nahi deta, ab jab sample poori tarah complete hai. Yahi wo headline hai jo PDF 8/9/10 mein baar-baar reference kiya gaya — 'no satellite-detectable effect, checked two ways, across two proxies.' Is PDF ka poora point ye dikhana tha ki ye headline, exact kaunsi code-lines se, kaise compute hota hai — aur wo lines khud, apni history mein, is study ka sabse dramatic reversal chhupaye baithi hain.

Part B — compute_border_distance.py: Har Gaon Ki Border Se Doori

Docstring — Ek Honest Disclaimer Upfront

```
"""
Distance from each village to the nearest India boundary segment (Natural
Earth Admin 0 lines, so simplified in the disputed LAC sector - not a claim).
Nearest segment isn't always China - can be Nepal/Bhutan/Myanmar too.
Using geodesic distance (WGS84) instead of a single UTM zone since villages
span 77-97 deg E and a fixed zone overstated distance for the far-east ones.
"""
```

Purpose: Ye docstring, sirf char lines mein, do zaroori, sensitive disclaimers upfront rakhta hai — dono baad mein Entry 6 aur Entry 18 mein hi discover hue the, lekin ab (final, corrected state mein) shuru se hi likhe hain. Pehla: is study ka 'border' ek Natural Earth cartographic-simplification hai, disputed LAC (Line of Actual Control) ka koi official ya legal claim nahi — ek genuinely zaroori caveat, kyunki India-China border ka koi ek internationally-agreed alignment hai hi nahi. Doosra: 'nearest border' hamesha China nahi hota — kuch villages Nepal, Bhutan, ya Myanmar ke zyada paas hain. Teesra: geodesic distance (globe ki curvature ko dhyaan mein rakh kar) use ho rahi hai, ek single flat-map projection (UTM) ki jagah, kyunki villages 77 se 97 degree East tak faili hui hain — itni badi range ke liye, ek single UTM zone accurate nahi reh sakta. Syntax: Multi-line docstring, teen alag methodological caveats, ek saath.

Imports Aur GEOD Constant

```
import geopandas as gpd
import pandas as pd
from pyproj import Geod
from shapely.ops import nearest_points

GEOD = Geod(ellps="WGS84")
```

Purpose: geopandas — pandas ka geographic-extension, jo coordinates ko 'geometry' objects (points, lines, polygons) ki tarah handle karta hai, PDF 7 mein bhi dekha gaya. pyproj se Geod — ek class jo Earth ke ellipsoid-shape (bilkul gol nahi, thoda chapta) ko dhyaan mein rakh kar, do coordinates ke beech *asli*, *geodesic* distance nikaalti hai — sirf flat-map par seedhi-rekha-doori nahi. shapely.ops se nearest_points — do geometries ke beech, sabse paas wale do points nikaalne wala function. GEOD = Geod(ellps="WGS84") — ek reusable object banaya jaata hai, WGS84 ellipsoid-model ke saath (yehi standard model hai jo GPS/lat-long coordinates use karte hain), taaki poori file mein isi ek object se distance-calculations ho sakein. Syntax: Standard geospatial imports. Geod(ellps="WGS84") — class-instantiation, ek keyword-argument (ellps) ke saath, jo batata hai kaunsa Earth-model use karna hai.

EXAMPLE — UTM Se Geodesic Tak — Entry 18 Ki Poori Kahani

Ye GEOD/pyproj-based approach, is file ka *current*, fixed version hai. Original version (Entry 6) mein, saari villages ek single UTM zone (44N, jiska center-meridian 81°E hai) mein reproject karke, seedhi-rekha planar-distance nikaali jaati thi — kaam karta tha, aur numbers 'plausible' dikhte the (0.1 se 69.4 km). Lekin Entry 18 mein, ek external review ne poocha: kya ek single UTM zone, itni badi east-west range (77°E se 97°E, matlab 20 degree se zyada) ke liye sahi hai? Check karne par, haan, ye ek real bias tha — jitna zyada east (Zone 44N ke center-meridian se door), utna zyada UTM projection distance ko *overstate* karta tha: Uttarakhand mein negligible, lekin Arunachal Pradesh ki sabse-east villages mein ~3% tak (worst case, ek village 2km se zyada off). Fix: nearest-point dhoondhna ab unprojected lon/lat mein hota hai (ye sirf 'kaunsa point sabse paas hai' decide karne ke liye kaafi accurate hai, is scale par), lekin actual reported distance ab pyproj.Geod se geodesically compute hoti hai — koi zone-dependency hi nahi, isliye poore study-area mein equally correct. Fix ke baad, saare H3 numbers thode se hile (jaise full-year NDBI-proximity rho 0.043→0.049), lekin koi bhi test significant-se-null ya null-se-significant nahi hua — ek genuine improvement, bina kisi conclusion ko palte.

Village Points Aur Boundary Lines Load Karna

```
villages = pd.read_csv("data/processed/border_optics_master_villages.csv")
villages_gdf = gpd.GeoDataFrame(
    villages,
    geometry=gpd.points_from_xy(villages["longitude"], villages["latitude"]),
    crs="EPSG:4326",
)

boundary = gpd.read_file("data/raw/ne_10m_admin_0_boundary_lines_land/ne_10m_admin_0_boundary_lines_land.shp")
```

Purpose: Master village-list padhi jaati hai (258 rows), aur ek GeoDataFrame banaya jaata hai — pandas ka normal dataframe, plus ek special 'geometry' column jisme har row ka apna Point-object hota hai (gpd.points_from_xy(longitude, latitude) se banaaya gaya). crs="EPSG:4326" batata hai ki ye coordinates kis 'coordinate reference system' mein hain — EPSG:4326 wahi standard hai jo plain lat/long (WGS84) use karta hai, GPS jaisa. Fir Natural Earth ki boundary-lines shapefile padhi jaati hai — ek special geographic file-format (.shp) jo real-world country-boundary lines rakhta hai. Syntax: Ye is PDF ki file mein module-level code hai (kisi function ke andar nahi — script top-se-bottom seedhe chalti hai, function-definitions ki jagah). gpd.GeoDataFrame(data, geometry=..., crs=...) — normal dataframe ko ek geographic-dataframe mein badalna. gpd.points_from_xy(x_col, y_col) — do numeric columns se, ek saath, Point-geometries ki ek poori series banata hai. gpd.read_file(path) — shapefile jaisi geographic files padhne ka standard tareeka.

India Ke Boundary-Segments Chunna — Ek Defensive Column-Check

```
print("Boundary file columns:", list(boundary.columns))
india_segments = boundary[
    (boundary.get("ADM0_A3_1") == "IND") | (boundary.get("ADM0_A3_2") == "IND")
    if "ADM0_A3_1" in boundary.columns else
    (boundary["ADM0_LEFT"] == "India") | (boundary["ADM0_RIGHT"] == "India")
].copy()
print(f"Found {len(india_segments)} India-related boundary segments")
```

Purpose: Ye block genuinely dhyaan se padhne layak hai, kyunki iska structure ek sikhi-hui-savdhaani (learned caution) dikhata hai. Comment (upar file mein hi) khud kehta hai — 'field names vary by Natural Earth version, check columns if this filter comes up empty.' Matlab, Natural Earth ke alag-alag downloads/versions mein, country-column ke naam alag ho sakte hain (kabhi ADM0_A3_1/ADM0_A3_2 — 3-letter country codes — kabhi ADM0_LEFT/ADM0_RIGHT — poore naam). Isliye ye ek *conditional expression* use karta hai — pehle check karo kaunse column-naam maujood hain, fir uske hisaab se sahi filter-condition chuno. Dono cases mein, matlab wahi hai: 'ye segment India ko chhoo raha hai, chahe India left-side par ho ya right-side par' (boundary-lines dataset mein har line do countries ke beech hoti hai,

isliye 'left'/'right' naming). Syntax: `list(boundary.columns)` — column-naam ek list mein, print karne ke liye. Ye ek *conditional expression* hai (A if condition else B), bilkul jaisa PDF 9 mein f-string ke andar dekha tha, lekin yahan ek poori boolean-mask hi conditionally chuni ja rahi hai — agar "ADM0_A3_1" column exist karta hai, ek expression use karo, warna doosra. `boundary.get("col")` — safe column-access, agar column na ho toh crash nahi, None milega. `|` — boolean OR, do masks ko jodne ke liye (pandas mein `or` keyword nahi, `|` symbol use hota hai row-wise operations ke liye).

EXAMPLE — Ye Defensive-Check Khud Entry 6 Ki History Se Aaya

Ye if "ADM0_A3_1" in `boundary.columns` else ... pattern, koi generic 'best practice' nahi hai jo shuru se yahan tha — ye Entry 6 ke apne experience se seekha gaya tha, jab pehli baar Natural Earth ki file download ki gayi thi aur uska schema expected se alag nikla (column-naam `ADM0_LEFT/ADM0_RIGHT` the, kisi doosre naam ke bajaye). Us waqt sirf ek hardcoded filter likha gaya hota — lekin agar kabhi dataset ka koi naya version download ho, jisme column-naam alag ho, ye code chup-chaap empty result de sakta tha (crash bhi nahi karta, sirf 0 segments milte, aur aage saari doori-calculations galat ho jaatin). Ye dual-check, us risk ko permanently address karta hai — chahe kaunsa bhi Natural Earth version ho, dono jaane-pehchaane schema-versions mein se kisi ek se match ho jaayega.

other_country() — Har Segment Ka 'Doosra Desh' Nikaalna

```
def other_country(row):  
    left, right = row.get("ADM0_LEFT"), row.get("ADM0_RIGHT")  
    return right if left == "India" else left  
  
india_segments["other_country"] = india_segments.apply(other_country, axis=1)
```

Purpose: Har boundary-segment do countries ke beech hota hai — is function ka kaam hai 'India' ko chhod kar, doosra desh kaun hai, wo nikaalna. Agar left-side 'India' hai, toh right-side wala doosra desh hai; warna (matlab right-side 'India' hai), left-side wala doosra desh hai. Ye naya column, other_country, aage ye batayega ki har gaon ka nearest-border asal mein kis desh ke saath hai — China, Nepal, Bhutan, ya Myanmar. **Syntax:** Function jo ek poori row leta hai (na ki ek single value), kyunki isse do columns (ADM0_LEFT, ADM0_RIGHT) ek saath chahiye. row.get("col") — safe access. right if left == "India" else left — conditional expression, do variables mein se ek chunta hai. .apply(function, axis=1) — dataframe ke har row par function chalana (axis=1 = row-wise; axis=0 hota toh column-wise hota), result ek naye column mein store karna.

border_union_4326 — Saare Segments Ko Ek Geometry Mein Jodna

```
border_union_4326 = india_segments.union_all()
```

Purpose: Ye ek chhoti, lekin zaroori line hai. Abhi tak, India ki boundary, alag-alag chhote-chhote segments (27 segments, Entry 18 ke hisaab se) ki tarah stored thi. .union_all() in sab segments ko jodkar, ek single, poori boundary-geometry banata hai — taaki aage 'is point se sabse paas border kahan hai' poochte waqt, saare 27 segments ko ek-ek karke check karne ki zaroorat na ho, seedha ek combined-shape se poocha ja sake. **Syntax:** .union_all() — geopandas ka method, ek GeoSeries ke saare geometries ko ek single geometry mein merge karta hai (jaise 27 alag lines ko ek poore, connected shape mein jodna).

nearest_border_km_and_country() — Is File Ka Core Function

```
def nearest_border_km_and_country(point):
    nearest = nearest_points(point, border_union_4326)[1]
    _, _, dist_m = GEOD.inv(point.x, point.y, nearest.x, nearest.y)
    seg_idx = india_segments.distance(nearest).idxmin()
    country = india_segments.loc[seg_idx, "other_country"]
    return pd.Series({"distance_to_border_km": dist_m / 1000, "nearest_border_country":
country})
```

Purpose: Ye function, ek gaon ke point ke liye, do cheezein deta hai — kitni doori hai border se (km mein), aur kaunse desh ka border sabse paas hai. Pehli line: `nearest_points(point, border_union_4326)` do points deta hai (input point khud, aur uska sabse-paas wala point border-geometry par) — [1] se doosra wala (border par wala point) liya jaata hai. Doosri line: `GEOD.inv(...)` — ye Geod object ka core function hai, jo do lat/lon points ke beech geodesic distance (aur direction) nikaalta hai; ismein teen values return hoti hain (forward-azimuth, back-azimuth, distance-in-meters), lekin sirf teesri (`dist_m`) chahiye, isliye pehli do `_` se discard ki jaati hain. Teesri line: ab pata karna hai *kaunsa specific segment* ye nearest-point tha (taaki uska `other_country` mil sake) — `india_segments.distance(nearest)` har segment ki, us nearest-point se, doori nikaalta hai, aur `.idxmin()` sabse-kam-doori-wale segment ka index deta hai. Aakhir mein, `.loc[seg_idx, "other_country"]` se us segment ka country-naam nikaala jaata hai, aur dono results — distance (km mein, meters ko 1000 se divide karke) aur country — ek `pd.Series` mein wapass kiye jaate hain. Syntax: `nearest_points(geom1, geom2)` — Shapely function, ek tuple deta hai (do points), [1] se doosra element access karna. `GEOD.inv(lon1, lat1, lon2, lat2)` — teen-value tuple return karta hai; `_, _, dist_m = ...` — tuple-unpacking jisme pehli do values jaan-bujh kar discard ki ja rahi hain (underscore ek convention hai 'ye value chahiye nahi'). `.distance(point)` — GeoSeries ka method, har geometry ki us point se doori deta hai. `.idxmin()` — sabse chhoti value ka *index* deta hai (`.min()` se alag, jo sirf value deta, position nahi). `.loc[index, "col"]` — label-based single-cell access. `pd.Series({...})` — ek dictionary se ek Series banana, function se multiple named-values ek saath return karne ke liye.

EXAMPLE — Distance Aur Country, Dono Ek Function Se Kyun

Ye design-choice — ek hi function se do related values return karna — efficient hai kyunki dono answers ko usi ek 'nearest point' calculation ki zaroorat hai. Agar do alag functions hote (ek distance ke liye, ek country ke liye), `nearest_points()` aur `GEOD.inv()` jaisi mehengi geometric-calculations, har gaon ke liye, do baar chalti — ek baar distance ke liye, ek baar country ke liye. Ek function se dono return karna, compute ko waste hone se bachaata hai.

Har Village Par Apply Karna, Aur Save Karna

```
nearest_info = villages_gdf.geometry.apply(nearest_border_km_and_country)

result = pd.concat([villages, nearest_info], axis=1)
result.to_csv("data/processed/border_optics_master_villages_with_distance.csv",
index=False)
print(f"\nSaved {len(result)} villages with border distance to
data/processed/border_optics_master_villages_with_distance.csv")
```

Purpose: `villages_gdf.geometry.apply(nearest_border_km_and_country)` — har gaon ke geometry-point par, ye function ek-ek karke chalta hai. Kyunki function ek `pd.Series` (do values ke saath) return karta hai, `.apply()` khud-b-khud in sab Series ko jodkar ek naya, do-column dataframe bana deta hai (`distance_to_border_km`, `nearest_border_country`). Fir `pd.concat([villages, nearest_info], axis=1)` — original village-data aur ye naye do columns, side-by-side jode jaate hain (columns add karna, rows nahi). Aakhir mein, ye enriched result CSV mein save hota hai — yahi wo file hai jo aage H3 test (PDF 13) use karega. Syntax: `.apply(function)` ek `GeoSeries` par, jab function ek Series return karta hai, result multiple columns ban jaata hai (single-value return hota toh ek hi column banta). `pd.concat([df1, df2], axis=1)` — `axis=1` ka matlab hai columns ko side-by-side jodo (`axis=0` hota toh rows ko upar-neeche stack karta).

```
print(result["distance_to_border_km"].describe())
print("\nNearest-border-country breakdown (core sample):")
print(result.loc[result["is_core_sample"] == True,
"nearest_border_country"].value_counts())
```

Purpose: Aakhri do prints, sanity-checking ke liye — distance-column ka summary (min/max/mean, etc.), aur core-sample ke liye, kitni villages kaunse desh ke sabse paas hain, ye breakdown. Syntax: `.describe()`, familiar. `result.loc[mask, "col"]` — rows filter karke, ek specific column select karna, ek hi `.loc[]` call mein. `.value_counts()` — har unique value (yahan, har desh) kitni baar aata hai, ye ginta hai.

Final State — 'Border' Ka Matlab, Is Study Mein

Nearest Country	Villages (core sample, n=251)	% of Sample
China	220	88%
Bhutan (mostly Tawang, West Kameng)	19	8%
Nepal (North Sikkim, Pithoragarh)	8	3%
Myanmar (Anjaw)	4	2%

Ye breakdown, Entry 18 mein pehli baar directly count kiya gaya — pehle ye assume kiya jaata tha ki is study ka 'border distance' matlab 'LAC/China distance' hai, lekin code khud kabhi specifically China check nahi karta tha, sirf 'nearest India-boundary, kisi bhi desh se.' Ye koi bug nahi tha — code sahi kaam kar raha tha jo usse likha gaya tha karne ke liye — lekin ye utni precisely disclose nahi hua tha jitna hona chahiye tha. Ab, is poori research mein har jagah, ye explicitly likha jaata hai: 88% sample ke liye ye genuinely China/LAC-proximity hai, lekin baaki 12% ke liye ye ek broader 'international-border-proximity' hai, LAC-specific nahi. Ek genuine, un-fixed inconsistency bhi disclose ki gayi hai: `select_control_villages.py` (PDF 7) apni khud ki, purani UTM44N-based distance-logic still use karta hai (sirf ek soft candidate-selection filter ke roop mein) — ise fully consistent karne ke liye control-list ko dobara generate karna padta, jo live Overpass access maangta hai, isliye flagged hai, silently chhupaya nahi gaya.

Is PDF mein humne dekha: `analyze_results.py`, ek consolidation-story ke saath shuru hota hai (do near-duplicate scripts ko ek `--window-flag` script mein jodna, before koi asli bug bane); iska core Wilcoxon-test block (dropna se `valid_ndbi` banana) hi wo exact jagah hai jahan is study ka sabse famous number — 154 se 251 villages — decide hota tha, Entry 5 ke Sikkim-summer-zero-images se lekar Entry 21 ke archive-timing-shak tak lekar Entry 22 ke fresh-same-day-fix tak; current, corrected state mein, chaaron H1 combinations (dono windows, dono proxies) null hain; `compute_border_distance.py` apna khud ka fix-history rakhta hai — UTM44N se geodesic tak (Entry 18, jitna east utna zyada bias tha, ab har jagah equally accurate), aur ek honest disclosure ki 'border' hamesha China nahi hota (88% China, baaki Bhutan/Nepal/Myanmar); aur is file ka defensive column-name-check khud Entry 6 ke ek real-world surprise se seekha gaya tha.

Agla PDF (PDF 12) H4 DiD model (did_model.py) dekhega — is study ka doosra bada reversal, control-group contamination wala. Jab bhi tayyar ho, bata dena.